

Assignment - 2 C++

Section - A

Q.1 Explain the concept of dynamic creation of variables in C++.

Ans Dynamic creation of variables means allocating memory during program execution instead of compile time. In C++, this is done using the new operator. The memory allocated remains available until it is released using the delete operator. This helps in efficient memory usage when the required size is not known in advance.

Q.2 Discuss how arrays and strings are used in C++ programming.

Ans Arrays are used to store multiple values of the same data type in contiguous memory location. They allow easy access using index values. Strings in C++ are used to store and manipulate sequences of characters. They can be handled using character arrays or the string class, which provides built-in functions for easy operation.

Q.3 What are classes and object in C++ and why are they important in programming.

Ans A class is a blueprint that defines data members and member functions, while an object is an instance of a class. Classes help in organizing code and implementing real-world concepts. Objects allow access to the class members. They are important for implementing Object Oriented Programming (OOP).

Q.4 Define encapsulation and explain its role in classes in C++.

Ans Encapsulation is the process of ~~now~~ wrapping data and function into a single unit called a class. It helps in hiding data using access ~~specifiers~~ specifiers like private and protected. Encapsulation improves data security and prevents unauthorized access. It also makes programs easier to maintain.

Q.5 Explain how to declare, define and access data member in C++ programming.

Ans Data members are declared inside a class. They are defined as part of the class structure. Access to data members is done through objects using the dot (.) operator. Access depends on the access specifier used (public, private, protected).

### Section - B

Q.1 Write a C++ program to demonstrate the use of inline functions and friend function.

Ans

```
#include <iostream>
using namespace std;
```

```
class Sample{
```

```
    int a;
```

```
public:
```

```
    Sample(int x) { a = x; }
```

```
    inline void show() {
```

```
        cout << "value of a:" << a << endl;
```

```
}  
  
friend void display (Sample s);  
};  
  
void display (Sample s) {  
    cout << "Friend Function Access : " << s.a << endl;  
}  
  
int main () {  
    Sample obj (10);  
    obj.show();  
    display (obj);  
    return 0;  
}
```

Q.2 WAP to create a class in C++ to represent a student with data members for name, roll number and marks. Include member function to set and get these values.

Ans

```
#include <iostream>  
using namespace std;  
  
class Student {  
    string name;  
    int roll;  
    float marks;  
public:  
    void setData (string n, int r, float m) {
```

```
name = n;  
roll = r;  
marks = m;  
}
```

```
void getData () {  
    cout << name << " " << roll << " " << marks;  
}
```

```
};
```

```
int main () {  
    Student s;  
    s.setData ("Aditya", 101, 85.5);  
    s.getData ();  
    return 0;  
}
```

Q.3 Write a C++ program to demonstrate the use of namespace in programming.

Ans # include <iostream>  
using namespace std;

```
namespace First {  
    int x = 10;  
}
```

```
namespace Second {  
    int x = 20;
```

}

```
int main () {  
    cout << First :: x << endl;  
    cout << Second :: x << endl;  
    return 0;  
}
```

}

### Section - C

Q.1 Differentiate between C++ Structure and classes in C++. Discuss the key differences in terms of data member, member function and access specifiers. Provide examples to illustrate your point.

| Basis          | Structure            | Class            |
|----------------|----------------------|------------------|
| Default access | Public               | Private          |
| Data security  | Less secure          | More secure      |
| OOP support    | Limited              | Full support     |
| Usage          | Simple data grouping | Complex programs |

E.g :- struct Data {  
 int x;  
};

```
class Data2 {  
    int y;  
};
```

Structures are mainly used for simple data storage, while classes

support full object-oriented programming.

Q.2 Explain the concept of programming paradigm in C++ and why they are important for organizing code.

Ans A programming paradigm is a style or approach used to design and write computer programs. It provides a structured way to think about problem solving and program organization. C++ is a multi-paradigm language, meaning it supports more than one programming paradigm.

C++ mainly supports procedural, object-oriented and generic programming paradigms. The procedural paradigm focuses on functions and step-by-step execution. The object-oriented paradigm organizes code using classes and objects, which helps in modeling real-world problems. The generic paradigm allows writing flexible and reusable code using templates.

Programming paradigms are important because they help in organizing code in a clear and systematic way. They improve reusability, readability and maintainability of programs. Using appropriate paradigms makes large and complex programs easier to develop, understand, and manage.

Q.3 Define a class in C++ for a student record, including data members such as student ID, name, and GPA. Implement member functions to input and display student details. Demonstrate the use of access specifiers (private, public) in the class.

Ans A Class in C++ is a user-defined data type that combine data members a member function into a single unit. To maintain data security, C++ provide access specifiers such as private and public. Data members declared as private cannot be accessed directly from outside the class, while public members can be accessed using objects.

In the following example, a Student class is defined with data members student ID, name and GPA, which are kept private to ensure data hiding. Public member functions are used to input and display student details. This demonstrates the concept of encapsulation, where data is protected and accessed only through authorized function.

```
#include <iostream>
using namespace std;
```

```
class Student {
private:
    int student ID;
    string name;
    float gpa;
}
```

```
void display () {
    cout << "Student ID" << student ID << endl;
    cout << "Name:" << name << endl;
    cout << "GPA:" << gpa << endl;
}
```

```
};
```

```
int main(){  
    Student s;  
    s.input();  
    s.display();  
    return 0;  
}
```

*Flow*